AI VIET NAM | All-in-One Course 2026

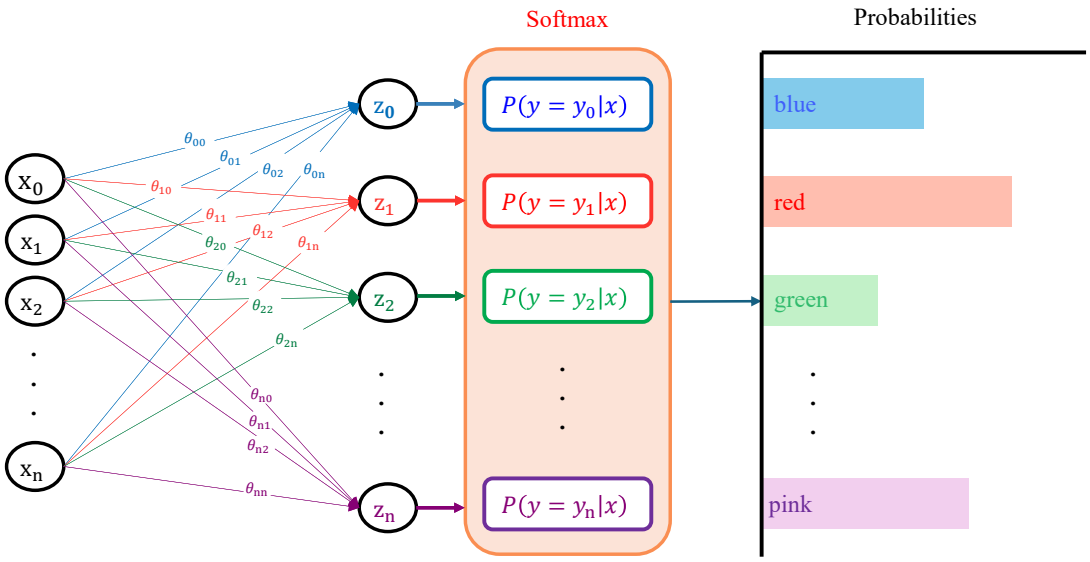
Exercise: Softmax Regression

Dương Đình Thắng **Nguyễn Đặng Duy Tân** **Đinh Quang Vinh**

1. Giới thiệu

Softmax Regression là một thuật toán học có giám sát, được thiết kế cho bài toán phân loại đa lớp. Điểm mấu chốt của thuật toán này là hàm Softmax, có chức năng biến các logit từ mô hình thành một phân bố xác suất trên tất cả các lớp [1]. Kết quả này cho phép ước lượng khả năng một mẫu thuộc về từng lớp, thay vì chỉ xếp mẫu đó vào đúng một lớp.

Trong bài viết này, chúng ta sẽ tìm hiểu công thức của hàm Softmax và cách điều chỉnh phân bố xác suất bằng tham số temperature. Sau đó, ta cài đặt Softmax Regression cho hai bài toán cụ thể. Bài toán thứ nhất là phát hiện giao dịch thẻ tín dụng gian lận, có hai lớp với tỉ lệ mẫu rất lệch. Bài toán thứ hai là phân loại cảm xúc ba lớp trên dữ liệu văn bản.



Hình 1: Softmax đặt ở đầu ra của mô hình, biến nhóm đầu ra tuyến tính (logit) thành xác suất của từng lớp, minh hoạ bằng các thanh ngang bên phải.

Mục lục

1. Giới thiệu **1**

2. Từ logit đến xác suất qua hàm Softmax **3**

2.1. Vector logit 3

2.2. Công thức Softmax 3

2.3. Ràng buộc chuẩn hoá 4

2.4. Ví dụ tính tay 4

3. Điều chỉnh độ tập trung của phân bố bằng tham số temperature **6**

4. Xây dựng Softmax Regression cho bài toán phát hiện gian lận thẻ tín dụng **8**

4.1. Bước 1: Nạp thư viện và đọc dữ liệu 8

4.2. Bước 2: Tách đặc trưng, thêm bias và mã hoá nhãn 9

4.3. Bước 3: Chia dữ liệu, chuẩn hoá và cân bằng lớp bằng SMOTE 10

4.4. Bước 4: Cài đặt các hàm thành phần 12

4.5. Bước 5: Huấn luyện mô hình 13

4.6. Bước 6: Đọc biểu đồ huấn luyện 15

5. Xây dựng Softmax Regression cho bài toán phân loại cảm xúc **17**

5.1. Bước 1: Nạp thư viện và đọc dữ liệu 17

5.2. Bước 2: Tiền xử lý và vector hoá văn bản 18

5.3. Bước 3: Chuẩn bị dữ liệu và định nghĩa mô hình 20

5.4. Bước 4: Huấn luyện mô hình 22

6. Tài liệu tham khảo **25**

7. Phụ lục **26**

2. Từ logit đến xác suất qua hàm Softmax

Softmax Regression là thuật toán học có giám sát dùng cho phân loại đa lớp, trong đó đầu ra được diễn giải như một phân bố xác suất trên các lớp. Ý tưởng cốt lõi là biến các logit tuyến tính thành xác suất chuẩn hoá, cho phép so sánh và ra quyết định theo xác suất giữa nhiều lớp cùng lúc.

2.1. Vector logit

Cho vector đặc trưng $\mathbf{x} \in \mathbb{R}^d$, ma trận trọng số $\mathbf{W} \in \mathbb{R}^{K \times d}$ và vector bias $\mathbf{b} \in \mathbb{R}^K$. Mô hình tính

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b} \in \mathbb{R}^K, \qquad \mathbf{z} = (z_1, \ldots, z_K)^\top.$$

Mỗi thành phần z_i được gọi là logit của lớp thứ i, tức giá trị đầu ra tuyến tính của lớp đó, còn ở dạng chưa chuẩn hoá thành xác suất. Logit nhận mọi giá trị thực, không bị chặn trong một khoảng cố định, nên chưa diễn giải được thành xác suất. Lớp nào có logit lớn hơn thì được mô hình ủng hộ hơn, nhưng mức độ ủng hộ chưa có thang đo chung.

2.2. Công thức Softmax

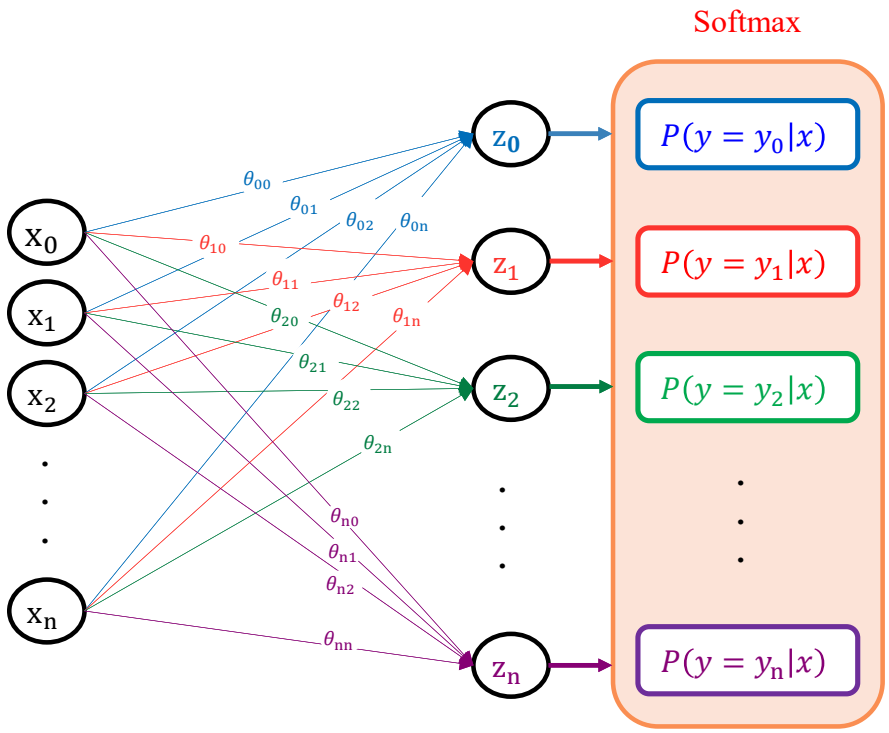
Hàm Softmax biến vector logit thành vector xác suất. Xác suất dự đoán cho lớp i là

$$p_i = \text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)}, \qquad i \in \{1, \ldots, K\}.$$

Công thức có hai bước. Hàm mũ đưa mọi logit về số dương và giữ nguyên thứ tự lớn nhỏ giữa các lớp. Phép chia cho tổng ở mẫu số đưa các số dương đó về cùng một thang, sao cho chúng cộng lại bằng 1.

Hai chỉ số trong công thức có vai trò khác nhau. Chỉ số i, gọi là chỉ số tự do, xác định thành phần đầu ra đang xét. Với mỗi i cố định, biểu thức trả về một số thực duy nhất trong $[0, 1]$. Khi i chạy từ 1 đến K, ta thu được toàn bộ vector $\mathbf{p} = (p_1, \ldots, p_K)^\top$. Chỉ số j, gọi là chỉ số chạy, chỉ xuất hiện bên trong tổng $\sum_{j=1}^{K}$, dùng để cộng qua tất cả các lớp ở mẫu số.

Mẫu số giống nhau trong công thức của mọi p_i, nên tăng một logit z_k bất kỳ sẽ làm tăng mẫu số. Với mọi lớp $i \neq k$, tử số $\exp(z_i)$ không đổi. Nhưng vì mẫu số đã lớn hơn, p_i giảm theo. Đây là cơ chế khiến các xác suất đầu ra ràng buộc lẫn nhau. Tăng xác suất của một lớp bắt buộc làm giảm xác suất của các lớp còn lại.



Hình 2: Mỗi đặc trưng x_j nối tới mọi logit z_i qua một trọng số riêng, tức một thành phần của ma trận $\mathbf{W}$. Hàm Softmax nhận cả nhóm logit và trả về xác suất $P(y = y_i \mid x)$ của từng lớp.

2.3. Ràng buộc chuẩn hoá

Nhờ mẫu số cộng qua tất cả các lớp, đầu ra luôn có tổng bằng 1 và mỗi thành phần không âm.

$$\sum_{i=1}^{K} p_i = 1, \qquad p_i \geq 0 \quad \forall i.$$

Hai tính chất này làm cho đầu ra dùng được như một phân bố xác suất. Tăng xác suất của một lớp buộc tổng xác suất các lớp còn lại giảm đúng bằng lượng đó. Nhờ vậy mô hình biểu diễn được sự cạnh tranh giữa các lớp, thay vì chấm điểm từng lớp một cách độc lập.

2.4. Ví dụ tính tay

Xét ba logit $\mathbf{z} = [2.0, 1.0, 0.1]$. Trước hết tính giá trị mũ của từng logit

$$\exp(2.0) = 7.3891, \quad \exp(1.0) = 2.7183, \quad \exp(0.1) = 1.1052.$$

Tổng ba giá trị này là mẫu số chung

$$S = \sum_{j=1}^{3} \exp(z_j) = 7.3891 + 2.7183 + 1.1052 = 11.2126.$$

Chia từng giá trị mũ cho S, ta được ba xác suất

$$p_1 = \frac{7.3891}{11.2126} \approx 0.6590, \qquad p_2 = \frac{2.7183}{11.2126} \approx 0.2424, \qquad p_3 = \frac{1.1052}{11.2126} \approx 0.0986.$$

Ba giá trị cộng lại bằng 1 đúng như ràng buộc. Trước khi chia cho S, hàm mũ đã khuếch đại chênh lệch 1.0 giữa hai logit đầu thành chênh lệch $7.3891 - 2.7183 = 4.6708$ giữa hai giá trị mũ. Phép chia cho S sau đó đưa chênh lệch này về 0.4166 trên thang xác suất bị chặn trong $[0, 1]$.

Mở rộng: ổn định số khi tính Softmax

Khi các logit lớn, giá trị $\exp(z_i)$ có thể tràn số. Với kiểu `float64` trong Python, $\exp(x)$ tràn khi x vượt quá khoảng 709, vì $e^{709} \approx 8.2 \times 10^{307}$ đã gần chạm giá trị lớn nhất mà kiểu này biểu diễn được, khoảng 1.8×10^{308}.

Cách xử lý là trừ đi logit lớn nhất trước khi tính mũ. Đặt $m = \max_j z_j$, ta tính

$$p_i = \frac{\exp(z_i - m)}{\sum_j \exp(z_j - m)}.$$

Phép trừ này không làm đổi kết quả. Cộng hay trừ cùng một hằng số cho mọi logit sẽ nhân cả tử và mẫu với cùng một thừa số e^{-m}, nên thương số giữ nguyên. Sau khi trừ, mọi số mũ đều không dương nên $\exp(z_j - m) \leq 1$ và không còn khả năng tràn.

Khi tính hàm mất mát cross-entropy, ta cần giá trị $\log \sum_j e^{z_j}$. Tổng bên trong cũng dễ tràn, nên dùng biến đổi log-sum-exp

$$\log \sum_j e^{z_j} = m + \log \sum_j e^{z_j - m}, \quad m = \max_j z_j.$$

Biến đổi giữ nguyên giá trị toán học và đưa mọi số mũ về không lớn hơn 1.

3. Điều chỉnh độ tập trung của phân bố bằng tham số temperature

Với cùng một vector logit, đôi khi ta muốn phân bố xác suất tập trung hơn vào một lớp, đôi khi lại muốn nó dàn đều hơn giữa các lớp. Tham số temperature $\tau > 0$ cho phép điều chỉnh độ tập trung này mà không cần huấn luyện lại mô hình. Ý tưởng là co giãn các logit trước khi chuẩn hoá

$$p_i(\tau) = \text{softmax}\left(\frac{\mathbf{z}}{\tau}\right)_i = \frac{\exp(z_i/\tau)}{\sum_{j=1}^{K} \exp(z_j/\tau)}, \qquad i = 1, \ldots, K.$$

Trường hợp $\tau = 1$ cho lại công thức Softmax thông thường. Khi τ tăng, phép chia $z_i \mapsto z_i/\tau$ thu hẹp khoảng cách giữa các logit. Hai logit cách nhau một khoảng Δ trước khi chia chỉ còn cách nhau Δ/τ sau khi chia, tức nhỏ hơn khi $\tau > 1$. Khoảng cách nhỏ hơn khiến các giá trị $\exp(z_i/\tau)$ gần nhau hơn, nên phân bố dàn đều hơn. Đại lượng đo mức dàn đều này là entropy

$$H\Big(p(\tau)\Big) = -\sum_{i=1}^{K} p_i(\tau) \log p_i(\tau),$$

và với $\mathbf{z}$ cố định, entropy tăng theo τ.

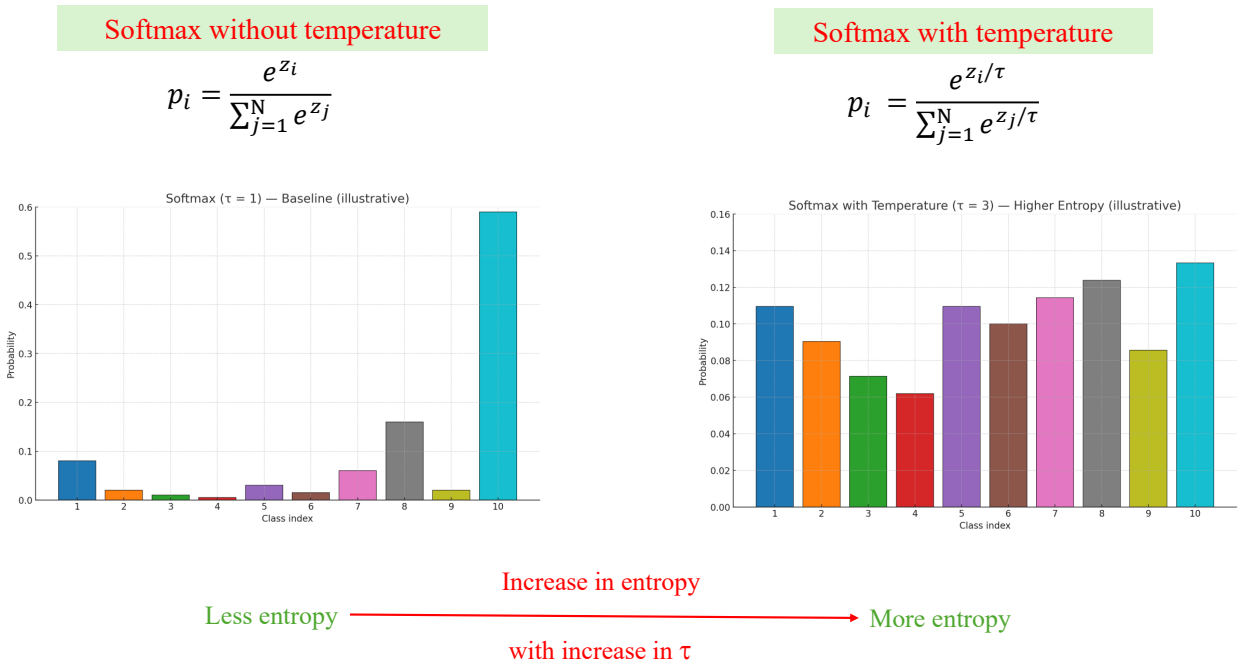
Hai trường hợp giới hạn cho thấy rõ vai trò của τ. Khi τ tiến về 0, phân bố tiến đến vector one-hot của lớp có logit lớn nhất

$$\lim_{\tau \to 0^+} p(\tau) = \mathbf{e}_{\arg\max_i z_i},$$

tức phân bố tập trung vào một lớp duy nhất và entropy đạt cực tiểu. Ngược lại, khi τ tiến ra vô cùng, phân bố tiến đến đều trên K lớp

$$\lim_{\tau \to \infty} p(\tau) = \left(\tfrac{1}{K}, \ldots, \tfrac{1}{K}\right),$$

tức mô hình không còn ưu tiên lớp nào và entropy đạt cực đại.



Hình 3: Cùng một vector logit trên 10 lớp cho hai phân bố khác nhau. Ở $\tau = 1$ bên trái, lớp thứ 10 chiếm gần 0.6 xác suất. Ở $\tau = 3$ bên phải, xác suất của mọi lớp nằm trong khoảng 0.06 đến 0.13, tức entropy đã tăng.

Mở rộng: chọn token kế tiếp trong mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn dùng Softmax để tạo phân bố $p(\tau)$ trên toàn bộ tập từ vựng, sau đó chọn token kế tiếp từ phân bố này. Bốn quy tắc chọn dưới đây khác nhau ở chỗ giữ lại bao nhiêu ứng viên trước khi chọn.

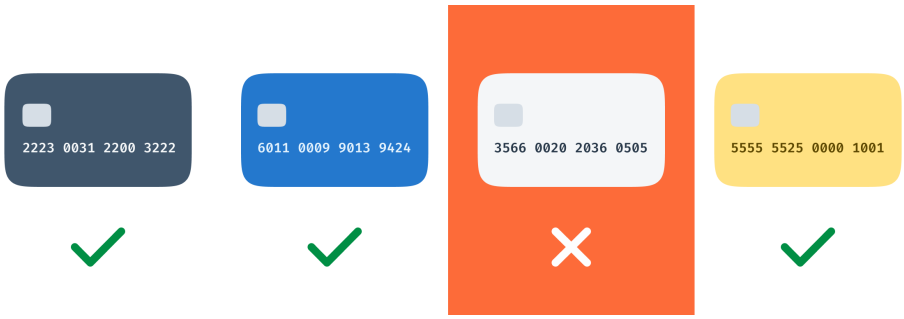
- **Greedy:** luôn chọn token có xác suất cao nhất. Cách này nhanh và cho kết quả lặp lại được, nhưng đầu ra ít đa dạng. Chọn token tốt nhất ở từng bước cũng không bảo đảm cả chuỗi có xác suất cao nhất.
- **Beam search:** giữ lại k chuỗi có tổng log-xác suất cao nhất sau mỗi bước, mở rộng chúng rồi lại giữ k chuỗi tốt nhất. Cách này thường cho chuỗi có xác suất cao hơn greedy, đổi lại tốn thêm bộ nhớ và thời gian theo k lần.
- **Top-k sampling:** giữ lại k token có xác suất cao nhất, chuẩn hoá lại phân bố trên tập con đó rồi lấy mẫu ngẫu nhiên [2]. Cách này loại bỏ các token hiếm ở phần đuôi phân bố, nhưng kích thước k cố định không phù hợp với mọi bước sinh.
- **Nucleus sampling (top-p):** giữ lại tập token nhỏ nhất có tổng xác suất không nhỏ hơn ngưỡng p, rồi lấy mẫu trong tập đó [3]. Số token được giữ thay đổi theo từng bước, nên khi mô hình đã chắc chắn thì tập ứng viên tự thu hẹp lại.

Bốn quy tắc đánh đổi giữa độ đa dạng và độ ổn định của đầu ra theo các mức khác nhau. Greedy và beam search cho kết quả ổn định nhất, hai cách lấy mẫu còn lại cho đầu ra đa dạng hơn.

4. Xây dựng Softmax Regression cho bài toán phát hiện gian lận thẻ tín dụng

Bài toán đặt ra là phát hiện giao dịch gian lận trong một tập giao dịch thẻ tín dụng. Đây là bài toán hai lớp, gồm giao dịch hợp lệ và giao dịch gian lận. Với hai lớp, Logistic Regression cùng hàm Sigmoid đã đủ để giải. Tuy nhiên, ta vẫn dùng Softmax Regression với $K = 2$ trong bài này, để làm quen với cách cài đặt sẽ dùng lại ở bài toán ba lớp tiếp theo. Cách cài đặt cho hai lớp và cho nhiều lớp là như nhau. Với $K = 2$, ta chỉ cần đối chiếu hai xác suất cộng lại bằng 1, nên kiểm tra lại kết quả trung gian dễ hơn.

Khó khăn chính của bài toán nằm ở tỉ lệ giữa hai lớp. Trong 284 807 giao dịch của tập dữ liệu [4], chỉ có 492 giao dịch gian lận, tức khoảng 0.17%. Một mô hình dự đoán mọi giao dịch đều hợp lệ vì thế đã đạt accuracy 99.83%. Mô hình này không phát hiện được trường hợp gian lận nào. Con số này cho thấy accuracy một mình chưa đủ để kết luận về mô hình trên dữ liệu mất cân bằng. Do đó, ngoài các bước huấn luyện thông thường, chúng ta cần thêm một bước cân bằng lại dữ liệu huấn luyện.



Hình 4: Minh hoạ tính chất nhị phân của bài toán. Mỗi đối tượng chỉ nhận một trong hai nhãn, hợp lệ (✓) hoặc gian lận (×), giống cách gán nhãn cho từng giao dịch bên dưới.

4.1. Bước 1: Nạp thư viện và đọc dữ liệu

```
!gdown --id 13Dlv9Ze336Od8hkd-R8Teb2nEGFRdeHP
!unzip "card_fraud_detection.zip"

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from imblearn.over_sampling import SMOTE

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Đọc dữ liệu
dataset_path = "creditcard.csv"
df = pd.read_csv(dataset_path)

# Thông tin tổng quan
df.info()
df.describe()
```

Tập dữ liệu `creditcard.csv` chứa các giao dịch thẻ tín dụng, mỗi dòng là một giao dịch và cột cuối là nhãn hợp lệ hay gian lận. Hai lệnh `df.info()` và `df.describe()` cho biết số dòng, kiểu dữ liệu của từng cột và các thống kê mô tả. Đây là căn cứ để quyết định các bước tiền xử lý tiếp theo.



Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V25	V26	V27	V28	Amount	Class
0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.189115	0.133558	-0.021053	149.62	0
0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.125895	-0.008983	0.014724	2.69	0
1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.139097	-0.055353	-0.059752	378.66	0
1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.221929	0.062723	0.061458	123.50	0
2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.502292	0.219422	0.215153	69.99	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
786.0	-11.881118	10.071785	-9.834783	-2.066656	-5.364473	-2.606837	-4.918215	7.305334	1.914428	...	0.213454	0.111864	1.014480	-0.509348	1.436807	0.250034	0.943651	0.823731	0.77	0
787.0	-0.732789	-0.055080	2.035030	-0.738589	0.868229	1.058415	0.024330	0.294869	0.584800	...	0.214205	0.924384	0.012463	-1.016226	-0.606624	-0.395255	0.068472	-0.053527	24.79	0
788.0	1.919565	-0.301254	-3.249640	-0.557828	2.630515	3.031260	-0.296827	0.708417	0.432454	...	0.232045	0.578229	-0.037501	0.640134	0.265745	-0.087371	0.004455	-0.026561	67.88	0
788.0	-0.240440	0.530483	0.702510	0.689799	-0.377961	0.623708	-0.686180	0.679145	0.392087	...	0.265245	0.800049	-0.163298	0.123205	-0.569159	0.546668	0.108821	0.104533	10.00	0
792.0	-0.533413	-0.189733	0.703337	-0.506271	-0.012546	-0.649617	1.577006	-0.414650	0.486180	...	0.261057	0.643078	0.376777	0.008797	-0.473649	-0.818267	-0.002415	0.013649	217.00	0

Hình 5: Một phần dữ liệu giao dịch, gồm các cột đặc trưng và cột nhãn ở cuối.

4.2. Bước 2: Tách đặc trưng, thêm bias và mã hoá nhãn

```python
# Chuyển DataFrame sang mảng NumPy
dataset_arr = df.to_numpy()

# Tách đặc trưng (X) và nhãn (y)
X, y = dataset_arr[:, :-1].astype(np.float64), dataset_arr[:, -1].astype(np.uint8)
print(X.shape)

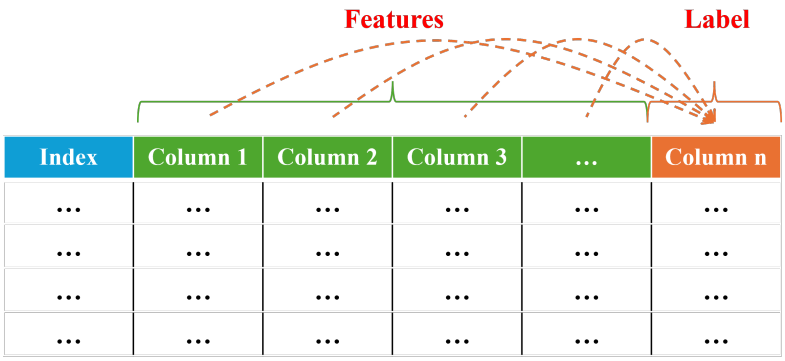
# Thêm thành phần bias vào ma trận đặc trưng
intercept = np.ones((X.shape[0], 1))
X_b = np.concatenate((intercept, X), axis=1)
print(X_b.shape)

# Mã hóa nhãn sang dạng one-hot
n_classes = np.unique(y, axis=0).shape[0]
n_samples = y.shape[0]

y_encoded = np.array([np.zeros(n_classes) for _ in range(n_samples)])
y_encoded[np.arange(n_samples), y] = 1
```

Bước này gồm ba thao tác:

- Chuyển dữ liệu từ `DataFrame` sang mảng NumPy, để các phép nhân ma trận ở phần sau viết được gọn hơn.
- Tách ma trận đặc trưng $\mathbf{X}$ khỏi vector nhãn $\mathbf{y}$, rồi thêm một cột toàn giá trị 1 vào trước $\mathbf{X}$ để gộp bias vào chung ma trận trọng số, ký hiệu là θ trong phần code.
- Mã hoá nhãn theo dạng one-hot, mỗi nhãn trở thành một vector dài K mang giá trị 1 ở vị trí lớp đúng, khớp với đầu ra xác suất dài K của Softmax.



Hình 6: Các cột đầu của bảng dữ liệu tạo thành ma trận đặc trưng **X**, còn cột cuối là nhãn **y** mà mô hình cần dự đoán.

4.3. Bước 3: Chia dữ liệu, chuẩn hoá và cân bằng lớp bằng SMOTE

```python
from imblearn.over_sampling import SMOTE

val_size = 0.2
test_size = 0.125
random_state = 2
is_shuffle = True

# Chia tập huấn luyện và validation
X_train, X_val, y_train, y_val = train_test_split(
    X_b, y_encoded,
    test_size=val_size,
    random_state=random_state,
    shuffle=is_shuffle
)

# Chia tiếp một phần từ tập huấn luyện để làm tập kiểm thử (test)
X_train, X_test, y_train, y_test = train_test_split(
    X_train, y_train,
    test_size=test_size,
    random_state=random_state,
    shuffle=is_shuffle
)

# Chuẩn hoá đặc trưng (bỏ qua cột bias)
normalizer = StandardScaler()
X_train[:, 1:] = normalizer.fit_transform(X_train[:, 1:])
X_val[:, 1:] = normalizer.transform(X_val[:, 1:])
X_test[:, 1:] = normalizer.transform(X_test[:, 1:])

# Cân bằng dữ liệu huấn luyện bằng SMOTE
smote = SMOTE(random_state=random_state, sampling_strategy="minority")
X_train_balanced, y_train_balanced = smote.fit_resample(
    X_train[:, 1:], np.argmax(y_train, axis=1)
)

# Thêm lại thành phần bias sau khi sinh thêm mẫu
intercept_balanced = np.ones((X_train_balanced.shape[0], 1))
X_train = np.concatenate((intercept_balanced, X_train_balanced), axis=1)
```

```
39
40 # Mã hóa lại nhãn theo dạng one-hot
41 y_train = np.zeros((y_train_balanced.shape[0], n_classes))
42 y_train[np.arange(y_train_balanced.shape[0]), y_train_balanced] = 1
43
44 # Kiểm tra lại phân phối lớp
45 print(f"Balanced training samples: {X_train.shape[0]}")
46 print(f"Class distribution: {np.bincount(y_train_balanced)}")
```

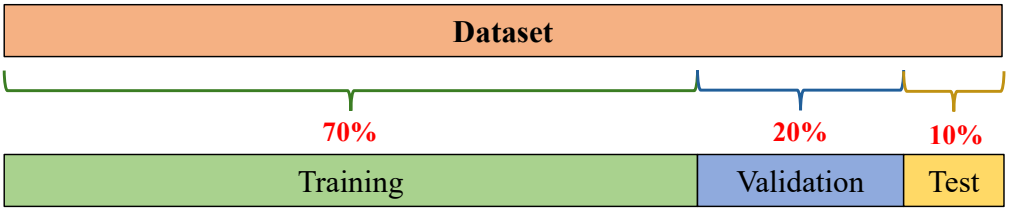
Đoạn code trên thực hiện bốn việc theo thứ tự.

Chia dữ liệu. Dữ liệu được tách thành ba phần. Tập huấn luyện dùng để cập nhật trọng số. Tập kiểm định dùng để theo dõi mô hình trong lúc huấn luyện. Tập kiểm thử chỉ dùng một lần ở cuối để báo cáo kết quả. Ba tập được giữ riêng biệt, nhờ đó kết quả cuối không chịu ảnh hưởng của các quyết định đã đưa ra dựa trên tập kiểm định.

Chuẩn hoá đặc trưng. Các cột đặc trưng được đưa về trung bình 0 và độ lệch chuẩn 1. Khi các đặc trưng có thang đo chênh lệch nhau, gradient theo các hướng cũng chênh lệch theo. Learning rate khi đó phải đủ nhỏ cho hướng dốc nhất, nên mô hình hội tụ chậm. Cột bias được giữ nguyên vì nó luôn bằng 1 và không có độ lệch chuẩn để chia.

Cân bằng lớp bằng SMOTE. Kỹ thuật này sinh thêm mẫu cho lớp thiểu số bằng cách nội suy giữa một mẫu thiểu số và các mẫu thiểu số gần nó nhất [5]. Khác với việc nhân bản mẫu cũ, cách nội suy tạo ra các điểm mới nằm giữa các mẫu đã có, nên vùng đặc trưng được xem là thuộc lớp thiểu số bao phủ rộng hơn, thay vì chỉ khoanh quanh vài điểm bị nhân bản. SMOTE chỉ áp dụng cho tập huấn luyện. Sinh thêm mẫu trên tập kiểm định hay tập kiểm thử sẽ làm kết quả đánh giá không còn phản ánh dữ liệu thật.

Khôi phục định dạng đầu vào. SMOTE chỉ chấp nhận nhãn dạng chỉ số lớp, không chấp nhận dạng one-hot, và chỉ nên sinh mẫu mới trên các cột đặc trưng, không áp dụng lên cột bias vốn luôn bằng 1. Vì vậy, trước khi gọi `smote.fit_resample`, cột bias được cắt khỏi **X** bằng `X_train[:, 1:]` và nhãn one-hot được đổi về chỉ số lớp bằng `np.argmax(y_train, axis=1)`. Sau khi SMOTE sinh thêm mẫu cho lớp thiểu số, cột bias được thêm lại và nhãn được mã hoá one-hot lần nữa, để khớp với định dạng mà các hàm `predict` và `compute_loss` ở Bước 4 yêu cầu.



Hình 7: Kết quả cuối cùng của hai lần chia: 70% dữ liệu dùng để huấn luyện, 20% dùng để kiểm định và 10% dùng để kiểm thử.

4.4. Bước 4: Cài đặt các hàm thành phần

Sáu hàm dưới đây tạo thành một vòng lặp huấn luyện hoàn chỉnh. Vai trò của từng hàm được giải thích ngay sau đoạn code.

Yêu cầu 1: Hoàn thành sáu hàm bên dưới tại các vị trí `***Your Code Here***`.

```python
def softmax(z):
    exp_z = ***Your Code Here***

    return ***Your Code Here***

def predict(X, theta):
    z = ***Your Code Here***
    y_hat = ***Your Code Here***

    return ***Your Code Here***

def compute_loss(y_hat, y):
    n = ***Your Code Here***

    return ***Your Code Here***

def compute_gradient(X, y, y_hat):
    n = ***Your Code Here***

    return ***Your Code Here***

def update_theta(theta, gradient, lr):
    return ***Your Code Here***

def compute_accuracy(X, y, theta):
    y_hat = ***Your Code Here***
    acc = ***Your Code Here***

    return acc
```

Hàm `softmax(z)` chuyển vector logit thành vector xác suất theo công thức

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Khi `z` là một batch, mỗi hàng là một mẫu, nên phép chia phải thực hiện theo từng hàng và mỗi hàng đầu ra có tổng bằng 1. Nên trừ đi giá trị lớn nhất của mỗi hàng trước khi tính mũ để tránh tràn số.

Hàm `predict(X, theta)` tính $z = X\theta$ rồi đưa kết quả qua `softmax`, trả về ma trận xác suất $\hat{y}$ có kích thước bằng số mẫu nhân số lớp.

Hàm `compute_loss(y_hat, y)` tính hàm mất mát cross-entropy

$$\mathcal{L} = -\frac{1}{NK}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\log(\hat{y}_{ik}),$$

với N là số mẫu trong batch và K là số lớp, tức NK là tổng số phần tử của ma trận nhãn (`n = y.size` trong code). Vì y ở dạng one-hot, tổng theo k chỉ giữ lại một số hạng ứng với lớp đúng. Hàm mất mát vì vậy đo mức độ mô hình gán xác suất thấp cho lớp đúng, và giảm về 0 khi xác suất của lớp đúng tiến về 1.

Hàm `compute_gradient(X, y, y_hat)` tính

$$\nabla_\theta = \frac{1}{NK} X^\top (\hat{y} - y),$$

chuẩn hoá theo cùng NK như `compute_loss`. Công thức này có cùng dạng với gradient của Logistic Regression. Đại lượng $\hat{y} - y$ là sai số dự đoán trên từng lớp, còn phép nhân với $X^\top$ phân bổ sai số đó về từng đặc trưng.

Hàm `update_theta(theta, gradient, lr)` cập nhật tham số theo

$$\theta \leftarrow \theta - \eta \nabla_\theta.$$

Giá trị η quyết định độ dài mỗi bước đi. Nếu η quá nhỏ, mô hình cần thêm nhiều epoch mới tới được vùng tối ưu. Nếu η quá lớn, bước cập nhật vượt qua điểm tối ưu và hàm mất mát dao động thay vì giảm.

Hàm `compute_accuracy(X, y, theta)` lấy `argmax` theo chiều lớp ở cả $\hat{y}$ và y, so sánh hai chỉ số rồi lấy trung bình số lần trùng nhau.

4.5. Bước 5: Huấn luyện mô hình

```
lr = 0.01
epochs = 30
batch_size = 1024
n_features = X_train.shape[1]

# Khởi tạo ngẫu nhiên tham số mô hình
np.random.seed(random_state)
theta = np.random.uniform(size=(n_features, n_classes))

# Lưu lịch sử huấn luyện
train_accs = []
train_losses = []
val_accs = []
val_losses = []

for epoch in range(epochs):
    train_batch_losses = []
    train_batch_accs = []
    val_batch_losses = []
    val_batch_accs = []

    for i in range(0, X_train.shape[0], batch_size):
        # Lấy mini-batch
        X_i = X_train[i:i+batch_size]
        y_i = y_train[i:i+batch_size]

        # Dự đoán và tính mất mát
        y_hat = predict(X_i, theta)
```

```
        train_loss = compute_loss(y_hat, y_i)

        # Tính gradient và cập nhật tham số
        gradient = compute_gradient(X_i, y_i, y_hat)
        theta = update_theta(theta, gradient, lr)

        # Ghi nhận kết quả batch huấn luyện
        train_batch_losses.append(train_loss)
        train_acc = compute_accuracy(X_train, y_train, theta)
        train_batch_accs.append(train_acc)

        # Tính mất mát và độ chính xác trên tập kiểm định
        y_val_hat = predict(X_val, theta)
        val_loss = compute_loss(y_val_hat, y_val)
        val_batch_losses.append(val_loss)

        val_acc = compute_accuracy(X_val, y_val, theta)
        val_batch_accs.append(val_acc)

    # Tính trung bình các chỉ số theo batch
    train_batch_loss = sum(train_batch_losses) / len(train_batch_losses)
    val_batch_loss = sum(val_batch_losses) / len(val_batch_losses)
    train_batch_acc = sum(train_batch_accs) / len(train_batch_accs)
    val_batch_acc = sum(val_batch_accs) / len(val_batch_accs)

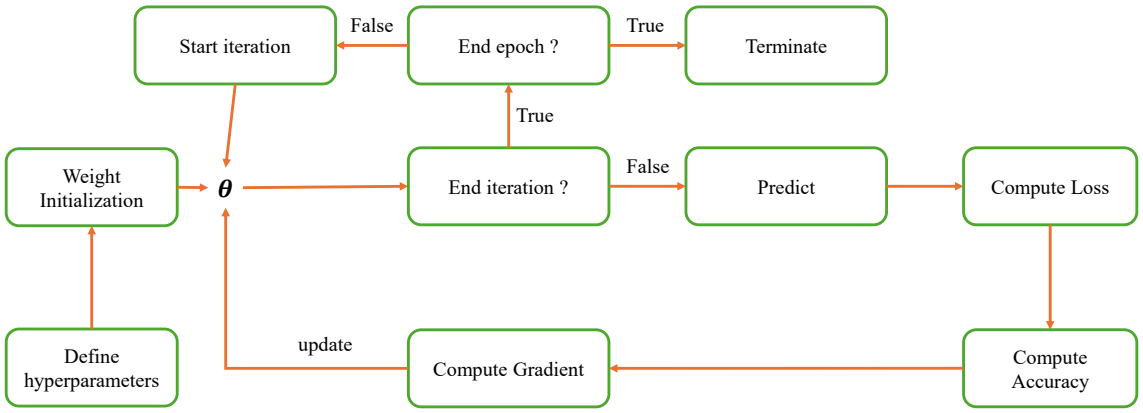
    # Lưu lại lịch sử qua từng epoch
    train_losses.append(train_batch_loss)
    val_losses.append(val_batch_loss)
    train_accs.append(train_batch_acc)
    val_accs.append(val_batch_acc)

    print(f"\nEPOCH {epoch + 1}:\tTraining loss: {train_batch_loss:.3f}\tValidation
                                        loss: {val_batch_loss:.3f}")
```

Vòng lặp huấn luyện có hai tầng. Tầng ngoài chạy qua 30 epoch, mỗi epoch duyệt hết dữ liệu một lần. Tầng trong cắt dữ liệu thành các mini-batch 1024 mẫu và cập nhật tham số một lần cho mỗi mini-batch.

Mini-batch cập nhật tham số theo từng lô nhỏ, thay vì theo từng mẫu hoặc theo toàn bộ dữ liệu cùng lúc. So với cập nhật theo từng mẫu, gradient trung bình trên 1024 mẫu bớt nhiễu hơn. So với cập nhật theo toàn bộ dữ liệu, mini-batch không phải đợi duyệt hết dữ liệu mới cập nhật, nên mỗi epoch đi được nhiều bước hơn.

Tham số θ được khởi tạo ngẫu nhiên với cùng một seed, nhờ đó chạy lại nhiều lần cho ra cùng kết quả. Mỗi epoch gồm nhiều mini-batch, kích thước mỗi batch là `batch_size`. Sau mỗi mini-batch, hàm mất mát và accuracy được ghi lại trên cả tập huấn luyện lẫn tập kiểm định, rồi lấy trung bình theo epoch. Theo dõi song song hai tập cho phép phát hiện quá khớp (overfitting). Đó là trường hợp hàm mất mát trên tập huấn luyện tiếp tục giảm trong khi trên tập kiểm định đã tăng trở lại.



Hình 8: Hai vòng lặp lồng nhau trong quá trình huấn luyện. Vòng trong duyệt từng mini-batch, gồm dự đoán, tính mất mát, tính gradient rồi cập nhật θ. Vòng ngoài lặp lại toàn bộ cho đến khi hết số epoch.

4.6. Bước 6: Đọc biểu đồ huấn luyện

```python
fig, ax = plt.subplots(2, 2, figsize=(12, 10))

# Training loss
ax[0, 0].plot(train_losses, color="green")
ax[0, 0].set(xlabel="Epoch", ylabel="Loss")
ax[0, 0].set_title("Training Loss")

# Validation loss
ax[0, 1].plot(val_losses, color="orange")
ax[0, 1].set(xlabel="Epoch", ylabel="Loss")
ax[0, 1].set_title("Validation Loss")

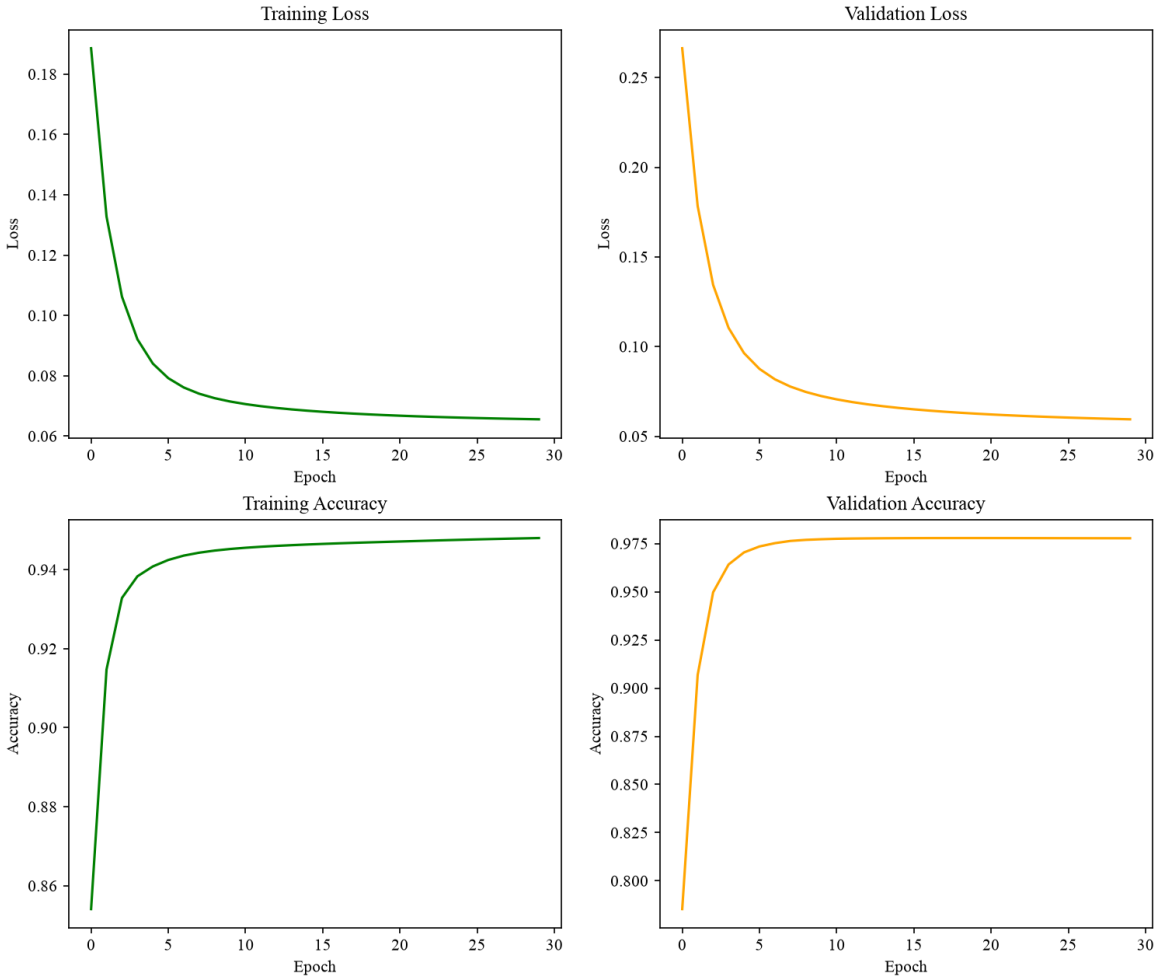
# Training accuracy
ax[1, 0].plot(train_accs, color="green")
ax[1, 0].set(xlabel="Epoch", ylabel="Accuracy")
ax[1, 0].set_title("Training Accuracy")

# Validation accuracy
ax[1, 1].plot(val_accs, color="orange")
ax[1, 1].set(xlabel="Epoch", ylabel="Accuracy")
ax[1, 1].set_title("Validation Accuracy")

plt.show()
```

Bốn biểu đồ trả lời hai câu hỏi khác nhau. Hàng trên vẽ hàm mất mát theo epoch, cho biết mô hình còn đang học hay đã dừng cải thiện. Hàng dưới vẽ accuracy, cho biết kết quả phân loại thực tế. Cột trái là tập huấn luyện, cột phải là tập kiểm định.

Khi đọc, hãy so hai biểu đồ cùng hàng với nhau. Nếu cả hai cùng giảm và có hình dạng giống nhau, mô hình đang học ổn định. Nếu đường mất mát của tập huấn luyện tiếp tục giảm trong khi đường của tập kiểm định đi ngang hoặc tăng trở lại, đó là dấu hiệu quá khớp và nên dừng sớm.

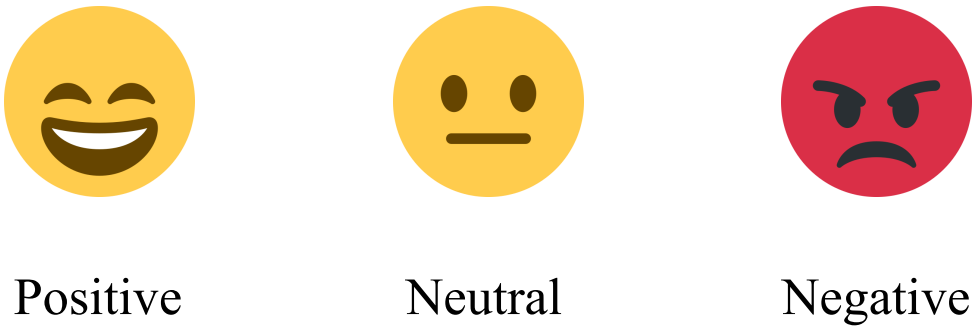


Hình 9: Hàm mất mát và accuracy trên tập huấn luyện và tập kiểm định qua 30 epoch.

Accuracy trên biểu đồ này tiến gần 100%, nhưng như đã thấy ở đầu mục này, con số đó một mình chưa xác nhận được mô hình phát hiện tốt giao dịch gian lận. Cần xem trực tiếp mô hình phát hiện đúng bao nhiêu giao dịch gian lận và bỏ sót bao nhiêu mới kết luận chắc chắn được.

5. Xây dựng Softmax Regression cho bài toán phân loại cảm xúc

Bài toán thứ hai là phân loại cảm xúc của các dòng trạng thái trên mạng xã hội thành ba nhãn tích cực, tiêu cực và trung lập. So với bài toán trước, bài này khác ở hai điểm. Thứ nhất, số lớp là ba nên đầu ra của Softmax có ba thành phần thay vì hai. Thứ hai, đầu vào là văn bản chứ không phải số, nên cần một bước biến văn bản thành vector đặc trưng trước khi đưa vào mô hình.



Hình 10: Ba nhãn cảm xúc mà mô hình cần phân biệt, ứng với $K = 3$ thành phần ở đầu ra của Softmax.

5.1. Bước 1: Nạp thư viện và đọc dữ liệu

```
!gdown --id 1gY-j4fRcnSNZYzBB-7lf-aaisduENDME
!unzip "../content/twitter_sentiment_analysis_3cls_dataset.zip"

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import re
import torch
import torch.nn as nn
import torch.optim as optim
import nltk
nltk.download("stopwords")

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from nltk.corpus import stopwords
from nltk.stem import SnowballStemmer

# Đọc dữ liệu gốc
dataset_path = "../content/Twitter_Data.csv"
df = pd.read_csv(dataset_path)
df

# Kiểm tra thông tin tổng quan
df.info()
df.describe()

# Kiểm tra và loại bỏ các dòng bị thiếu dữ liệu
```

```
null_rows = df.isnull().any(axis=1)
df[null_rows]
df = df.dropna()
```

Ở bước này, ta chuẩn bị dữ liệu trước khi tiền xử lý:

- Tập dữ liệu `Twitter_Data.csv` gồm hai cột, một cột chứa nội dung văn bản và một cột chứa nhãn cảm xúc.
- Thư viện `nltk` cung cấp danh sách từ dừng và bộ rút gọn từ về dạng gốc, dùng ở Bước 2.
- Lớp `TfidfVectorizer` biến văn bản thành vector số, còn thư viện `torch` dùng để dựng và huấn luyện mô hình ở các bước sau.
- Các dòng thiếu giá trị được loại bỏ bằng `dropna()`, vì một dòng thiếu nội dung hoặc thiếu nhãn sẽ gây lỗi ở bước vector hoá.

	clean_text	category
0	when modi promised “minimum government maximum...	-1.0
1	talk all the nonsense and continue all the dra...	0.0
2	what did just say vote for modi welcome bjp t...	1.0
3	asking his supporters prefix chowkidar their n...	1.0
4	answer who among these the most powerful world...	1.0
...	...	...
162975	why these 456 crores paid neerav modi not reco...	-1.0
162976	dear rss terrorist payal gawar what about modi...	-1.0
162977	did you cover her interaction forum where she ...	0.0
162978	there big project came into india modi dream p...	0.0
162979	have you ever listen about like gurukul where ...	1.0

162980 rows × 2 columns

Hình 11: Một phần dữ liệu gốc, gồm cột văn bản (chưa qua bước làm sạch ở Bước 2) và cột nhãn cảm xúc tương ứng.

5.2. Bước 2: Tiền xử lý và vector hoá văn bản

```
def text_normalize(text):
    # Chuyển toàn bộ về chữ thường
    text = text.lower()

    # Xoá tiền tố "RT" của các dòng được đăng lại
    text = re.sub(r"^rt[\s]+", "", text)

    # Loại bỏ đường dẫn (URLs)
    text = re.sub(r"https?:\/\/.*[\r\n]*", "", text)

```

```
    # Loại bỏ dấu câu và ký tự đặc biệt
    text = re.sub(r"[^\w\s]", "", text)

    # Loại bỏ stopwords
    stop_words = set(stopwords.words("english"))
    words = text.split()
    words = [word for word in words if word not in stop_words]
    text = " ".join(words)

    # Stemming (đưa từ về gốc)
    stemmer = SnowballStemmer("english")
    words = text.split()
    words = [stemmer.stem(word) for word in words]
    text = " ".join(words)

    return text

# Ví dụ minh hoạ
text = """We love this! Would you go?
#talk #makememories #unplug
#relax #iphone #smartphone #wifi #connect...
http://fb.me/6N3LsUpCu
"""
text = text_normalize(text)
text

# Áp dụng chuẩn hoá cho toàn bộ tập dữ liệu
df["clean_text"] = df["clean_text"].apply(lambda x: text_normalize(x))

# Biểu diễn văn bản bằng TF-IDF
vectorizer = TfidfVectorizer(max_features=2000)
X = vectorizer.fit_transform(df["clean_text"]).toarray()
```

Mô hình chỉ nhận đầu vào là số, nên văn bản phải qua hai giai đoạn. Giai đoạn thứ nhất làm sạch, giai đoạn thứ hai vector hoá.

Hàm `text_normalize` thực hiện sáu phép làm sạch theo thứ tự:

1. Chuyển về chữ thường, để hai dạng viết của cùng một từ không bị coi là hai từ khác nhau.
2. Xoá tiền tố `RT` của các dòng được đăng lại, vì không mang thông tin cảm xúc.
3. Xoá các đường dẫn, vì cũng không mang thông tin cảm xúc.
4. Xoá dấu câu và ký tự đặc biệt, để tách từ cho đúng.
5. Loại bỏ từ dừng, tức các từ xuất hiện ở hầu hết văn bản nên không phân biệt được nhãn.
6. Stemming, đưa các biến thể của một từ về cùng một dạng gốc.

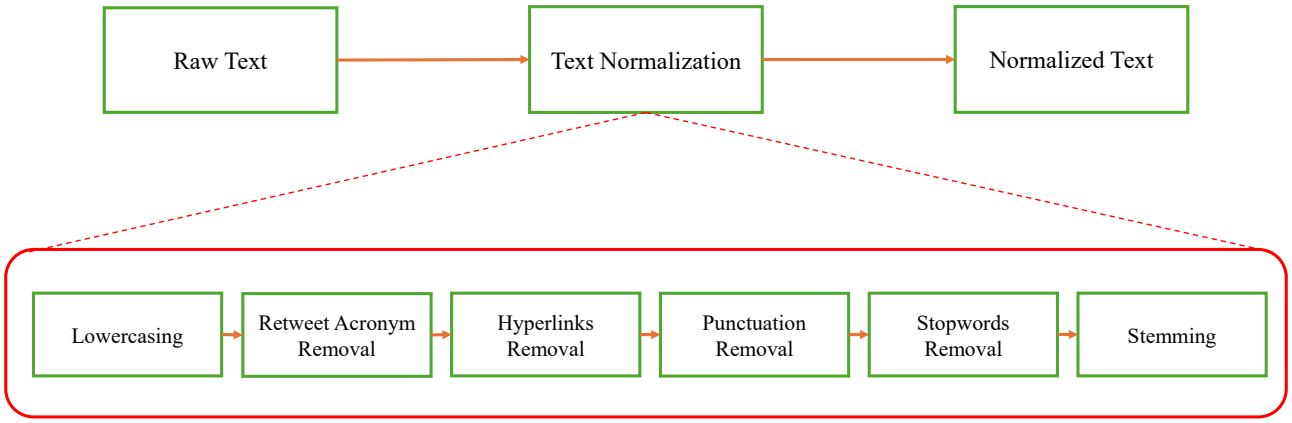
Bước vector hoá dùng TF-IDF, kết hợp tần suất xuất hiện của từ (term frequency) với nghịch đảo tần suất văn bản (inverse document frequency), đại lượng đo độ đặc trưng của một từ trong tập văn bản

$$\text{tfidf}(t) = \text{tf}(t) \times \left(\log \frac{1+M}{1+\text{df}(t)} + 1 \right),$$

trong đó tf(t) là số lần từ t xuất hiện trong văn bản đang xét, M là tổng số văn bản, còn df(t) là số văn bản có chứa từ t. Số hạng cộng thêm 1 ở tử và mẫu tránh chia cho 0 khi một từ xuất hiện ở mọi văn bản, đúng theo tham số mặc định của `TfidfVectorizer`. Sau khi tính, mỗi vector văn bản còn được chuẩn hoá về độ dài 1.

- tfidf(t) tăng theo tf(t) và giảm khi df(t) tăng, tức từ t được chấm điểm cao khi xuất hiện nhiều trong văn bản đang xét nhưng hiếm gặp ở các văn bản khác. Điểm số này không xét từ đó có liên quan đến nhãn cảm xúc hay không. Việc phân biệt nhãn thuộc về Softmax Regression ở bước huấn luyện sau.
- Tham số `max_features=2000` giữ lại 2000 từ có điểm cao nhất, nên mỗi văn bản trở thành một vector 2000 chiều.

Bước này gọi `fit_transform` trên toàn bộ dữ liệu, trước khi chia tập ở Bước 3. Do đó df(t) được tính luôn trên cả tập kiểm định và kiểm thử, khác với nguyên tắc chỉ dùng tập huấn luyện đã áp dụng cho SMOTE ở bài toán trước.



Hình 12: Sáu phép làm sạch bên trong hàm `text_normalize`, áp dụng lần lượt từ trái sang phải để đưa văn bản gốc về văn bản đã chuẩn hoá.

5.3. Bước 3: Chuẩn bị dữ liệu và định nghĩa mô hình

Từ bước này, mô hình được dựng bằng PyTorch thay vì NumPy thuần. Thư viện tự tính gradient, nên phần cài đặt chỉ còn hai việc là mô tả kiến trúc và viết vòng lặp huấn luyện.

Yêu cầu 2: Hoàn thành phần chuẩn bị dữ liệu và lớp mô hình bên dưới.

```
n_classes = ***Your Code Here***
n_samples = ***Your Code Here***

# category nhận giá trị -1, 0, 1: cần cộng 1 trước khi ép kiểu
y = ***Your Code Here***
y = y.astype(np.uint8)
y_encoded = np.array(
    ***Your Code Here***
)
y_encoded[np.arange(n_samples), y] = 1
```

```

X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y_encoded, dtype=torch.float32)

val_size = 0.2
test_size = 0.125
random_state = 2
is_shuffle = True

X_train, X_val, y_train, y_val = train_test_split(
    ***Your Code Here***,
    test_size=***Your Code Here***,
    random_state=random_state,
    shuffle=is_shuffle
)

X_train, X_test, y_train, y_test = train_test_split(
    ***Your Code Here***,
    test_size=***Your Code Here***,
    random_state=random_state,
    shuffle=is_shuffle
)

class SoftmaxRegression(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(SoftmaxRegression, self).__init__()
        self.linear = nn.Linear(
            ***Your Code Here***,
            ***Your Code Here***,
            bias=True
        )

    def forward(self, x):
        return ***Your Code Here***

def compute_accuracy(y_hat, y_true):
    _, y_hat = torch.max(y_hat, dim=1)
    _, y_true = ***Your Code Here***

    correct = ***Your Code Here***
    accuracy = ***Your Code Here***

    return accuracy
```

Phần chuẩn bị dữ liệu lặp lại các thao tác đã làm ở bài toán trước:

- Số lớp và số mẫu quyết định kích thước ma trận nhãn one-hot.
- Nhãn gốc trong cột `category` nhận giá trị -1, 0, 1 (tiêu cực, trung lập, tích cực). Cộng thêm 1 để đưa về chỉ số lớp không âm 0, 1, 2 trước khi ép sang `uint8`, vì kiểu số nguyên không dấu này không biểu diễn được giá trị âm. Cả đặc trưng lẫn nhãn sau đó được chuyển sang tensor của PyTorch.
- Dữ liệu được chia thành ba tập theo cùng tỉ lệ như trước.

Lớp `SoftmaxRegression` chỉ có một tầng `nn.Linear` nhận vector 2000 chiều và trả về ba logit. Hàm `forward` trả thẳng logit chứ không gọi Softmax, vì hàm mất mát `CrossEntropyLoss` của PyTorch đã gộp sẵn Softmax vào bên trong.

Gọi Softmax hai lần sẽ làm hỏng phân bố xác suất:

- Softmax thứ hai nhận đầu vào là các xác suất đã nằm trong khoảng $(0, 1)$, thay vì logit gốc vốn có biên độ lớn hơn nhiều.
- Vì biên độ đầu vào nhỏ hơn, Softmax thứ hai cho ra phân bố dàn đều hơn mức đúng, khiến mô hình học sai.

Hàm `compute_accuracy` lấy chỉ số lớp có giá trị lớn nhất ở cả dự đoán và nhãn thật. Hai chỉ số được so sánh để đếm số lần trùng, sau đó chia cho tổng số mẫu.

5.4. Bước 4: Huấn luyện mô hình

Yêu cầu 3: Hoàn thành vòng lặp huấn luyện bên dưới.

```
lr = 0.1
epochs = 500
torch.manual_seed(random_state)
if torch.cuda.is_available():
    torch.cuda.manual_seed(random_state)

input_dim = X_train.shape[1]
output_dim = y_train.shape[1]

model = SoftmaxRegression(
    ***Your Code Here***
)
criterion = ***Your Code Here***
optimizer = ***Your Code Here***

train_losses = []
train_accs = []
val_losses = []
val_accs = []

for epoch in range(epochs):
    model.***Your Code Here***

    # Zero the gradients
    optimizer.zero_grad()

    # Forward pass
    y_hat = model(X_train)

    # Compute loss
    train_loss = ***Your Code Here***

    train_losses.append(train_loss.item())

    train_acc = compute_accuracy(y_hat, y_train)
    train_accs.append(train_acc)
```

```
    # Backward pass and optimization
    train_loss.***Your Code Here***
    optimizer.***Your Code Here***

    model.eval()
    # Forward pass for validation data
    with torch.***Your Code Here***:
        y_val_hat = model(X_val)

        # Compute validation loss
        val_loss = criterion(y_val_hat, y_val)
        val_losses.append(val_loss.item())

        val_acc = compute_accuracy(y_val_hat, y_val)
        val_accs.append(val_acc)

    print(f"\nEPOCH {epoch + 1}:\tTraining loss: {train_loss:.3f}\tValidation loss: {
                                        val_loss:.3f}")
```

Vòng lặp này dùng toàn bộ tập huấn luyện cho mỗi lần cập nhật, tức Batch Gradient Descent, khác với cách chia mini-batch ở bài toán trước. Mỗi epoch vì vậy chỉ đi đúng một bước, và đó là lý do số epoch ở đây là 500 thay vì 30.

Mỗi epoch gồm bốn thao tác theo thứ tự cố định. Trước hết, `optimizer.zero_grad()` xoá gradient của lần cập nhật trước, vì PyTorch mặc định cộng dồn gradient qua các lần gọi. Tiếp theo là lan truyền xuôi để tính logit và hàm mất mát. Sau đó, lan truyền ngược tính gradient theo từng tham số. Cuối cùng, bộ tối ưu áp dụng gradient để cập nhật trọng số.

Phần đánh giá trên tập kiểm định đặt trong khối `torch.no_grad()` và sau khi gọi `model.eval()`. Khối này tắt việc dựng đồ thị tính đạo hàm, nhờ đó tiết kiệm bộ nhớ và bảo đảm dữ liệu kiểm định không ảnh hưởng đến trọng số.

Sau khi huấn luyện xong, chúng ta vẽ lại hàm mất mát và accuracy theo epoch.

```
fig, ax = plt.subplots(2, 2, figsize=(12, 10))

# Training loss
ax[0, 0].plot(train_losses, color="green")
ax[0, 0].set(xlabel="Epoch", ylabel="Loss")
ax[0, 0].set_title("Training Loss")

# Validation loss
ax[0, 1].plot(val_losses, color="orange")
ax[0, 1].set(xlabel="Epoch", ylabel="Loss")
ax[0, 1].set_title("Validation Loss")

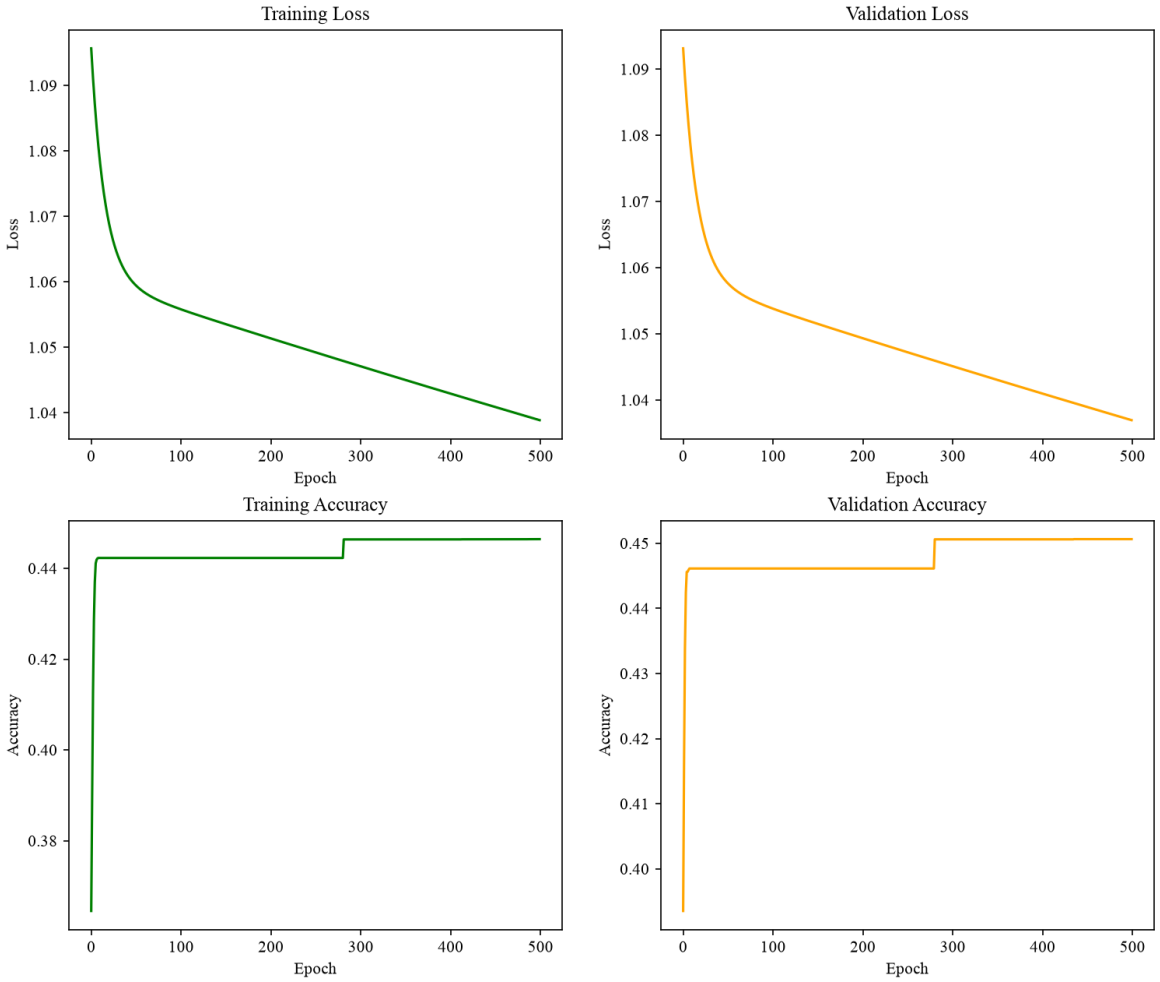
# Training accuracy
ax[1, 0].plot(train_accs, color="green")
ax[1, 0].set(xlabel="Epoch", ylabel="Accuracy")
ax[1, 0].set_title("Training Accuracy")

# Validation accuracy
```

```
ax[1, 1].plot(val_accs, color="orange")
ax[1, 1].set(xlabel="Epoch", ylabel="Accuracy")
ax[1, 1].set_title("Validation Accuracy")

plt.show()
```

Cách đọc bốn biểu đồ giống như ở bài toán trước. Ở đây, accuracy của cả hai tập đều sớm đi ngang ở mức thấp. Mức này gần như không đổi suốt phần còn lại, cho thấy mô hình học được rất ít từ đặc trưng TF-IDF với cấu hình này.



Hình 13: Hàm mất mát và accuracy trên tập huấn luyện và tập kiểm định qua 500 epoch.

6. Tài liệu tham khảo

[1] J. S. Bridle, "Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition", in *Neurocomputing: Algorithms, Architectures and Applications*, ser. NATO ASI Series, vol. 68, Berlin, Heidelberg: Springer, 1990, pp. 227–236. DOI: 10.1007/978-3-642-76153-9_28.

[2] A. Fan et al., "Hierarchical neural story generation", in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018, pp. 889–898. DOI: 10.18653/v1/P18-1082.

[3] A. Holtzman et al., "The curious case of neural text degeneration", in *International Conference on Learning Representations (ICLR)*, 2020.

[4] A. Dal Pozzolo et al., "Calibrating probability with undersampling for unbalanced classification", in *IEEE Symposium Series on Computational Intelligence (SSCI)*, 2015, pp. 159–166. DOI: 10.1109/SSCI.2015.33.

[5] N. V. Chawla et al., "SMOTE: Synthetic minority over-sampling technique", *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002. DOI: 10.1613/jair.953.

[6] D. R. Cox, "The regression analysis of binary sequences", *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 20, no. 2, pp. 215–242, 1958. DOI: 10.1111/j.2517-6161.1958.tb00292.x.

[7] I. Goodfellow et al., *Deep Learning*. Cambridge, MA: MIT Press, 2016.

[8] S. Ruder, "An overview of gradient descent optimization algorithms", *arXiv preprint arXiv:1609.04747*, 2016.

[9] G. Hinton et al., "Distilling the knowledge in a neural network", *arXiv preprint arXiv:1503.02531*, 2015.

[10] K. Spärck Jones, "A statistical interpretation of term specificity and its application in retrieval", *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972. DOI: 10.1108/eb026526.

7. Phụ lục

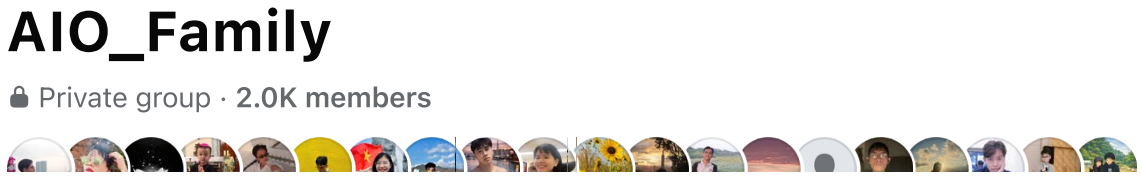
1. **Resources:** Các file code gợi ý (hint) và bài giải (solution) có thể được tải tại đây. (Lưu ý: Sau khi hết deadline của phần bài này, ad mới copy các nội dung bài giải vào đường dẫn).

2. **Rubric:**

Mục	Kiến thức	Đánh giá
1.	- Hạn chế của Logistic Regression khi số lớp lớn hơn hai. - Vai trò của hàm mũ và của phép chia cho tổng trong công thức Softmax. - Ràng buộc chuẩn hoá của vector xác suất đầu ra.	- Giải thích được vì sao dùng Sigmoid riêng cho từng lớp không cho ra một phân bố xác suất. - Nêu đúng vai trò của chỉ số tự do và chỉ số chạy trong công thức. - Chứng tỏ được tổng xác suất đầu ra bằng 1.
2.	- Cách tính Softmax bằng tay trên một vector logit. - Hiện tượng tràn số khi tính hàm mũ và cách trừ đi giá trị lớn nhất. - Biến đổi log-sum-exp.	- Tính đúng vector xác suất từ một vector logit cho trước. - Giải thích được vì sao trừ đi $\max_j z_j$ không làm đổi kết quả. - Nêu được trường hợp cần dùng log-sum-exp.
3.	- Tham số temperature và tác động của nó lên entropy của phân bố. - Hai trường hợp giới hạn khi τ tiến về 0 và khi τ tăng. - Bốn quy tắc chọn token trong mô hình ngôn ngữ lớn.	- Dự đoán đúng chiều thay đổi của phân bố khi tăng hoặc giảm τ. - Nêu được phân bố giới hạn ở hai đầu và entropy tương ứng. - Phân biệt được top-k và top-p về cách xác định tập ứng viên.
4.	- Tiền xử lý dữ liệu bảng gồm tách đặc trưng, thêm bias, mã hoá one-hot. - Chia dữ liệu thành ba tập và chuẩn hoá bằng `StandardScaler`. - Kỹ thuật SMOTE cho dữ liệu mất cân bằng.	- Cài đặt đúng các bước tiền xử lý trên tập giao dịch thẻ tín dụng. - Giải thích được vì sao chỉ áp dụng SMOTE cho tập huấn luyện. - Nêu được vì sao accuracy cao chưa đủ để kết luận trên dữ liệu mất cân bằng.
5.	- Sáu hàm thành phần gồm `softmax`, `predict`, `compute_loss`, `compute_gradient`, `update_theta`, `compute_accuracy`. - Công thức cross-entropy và gradient $\frac{1}{NK}X^\top(\hat{y}-y)$.	- Cài đặt đúng sáu hàm bằng NumPy thuần. - Giải thích được vì sao tổng theo lớp trong cross-entropy chỉ giữ lại một số hạng. - Nêu đúng ý nghĩa của $\hat{y}-y$ trong công thức gradient.
6.	- Huấn luyện theo mini-batch và vai trò của `batch_size`. - Cách đọc biểu đồ hàm mất mát và accuracy trên hai tập. - Dấu hiệu quá khớp.	- Huấn luyện được mô hình và đọc đúng kết quả qua từng epoch. - So sánh được mini-batch với cập nhật theo từng mẫu và theo toàn bộ dữ liệu. - Phát hiện được quá khớp từ khoảng cách giữa hai đường trên biểu đồ.

7.	- Tiền xử lý văn bản gồm chuẩn hoá, loại bỏ từ dừng và stemming. - Biểu diễn văn bản bằng TF-IDF và ý nghĩa của `max_features`.	- Cài đặt đúng hàm `text_normalize`. - Giải thích được vì sao TF-IDF hạ trọng số của các từ phổ biến. - Nêu được ảnh hưởng của `max_features` đến số chiều đặc trưng.
8.	- Dựng Softmax Regression bằng PyTorch với một tầng `nn.Linear`. - Quan hệ giữa `CrossEntropyLoss` và hàm Softmax. - Vòng lặp huấn luyện với `zero_grad`, `backward`, `step` và khối `torch.no_grad()`.	- Hoàn thành đúng lớp mô hình và vòng lặp huấn luyện. - Giải thích được vì sao `forward` trả về logit thay vì xác suất. - Nêu đúng thứ tự bốn thao tác trong một epoch và lý do của thứ tự đó.

3. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.



Hình 14: Hình ảnh group facebook AIO Q&A.